

12장 저장장치와 입출력 시스템

학습목표

- 저장장치의 특징과 디스크 스케줄링 기법에 대하여 학습한다.
- 디스크 장애 발생 시 복구에 사용되는 RAID의 작동 방식을 이해한다.
- 입출력 장치의 속도를 향상하는 다양한 기술을 살펴본다.

목차

1. 저장장치와 디스크 스케줄링
2. RAID
3. 입출력 시스템

01

저장장치와 디스크 스케줄링

01 저장장치와 디스크 스케줄링

1. 자기 디스크 기반 저장장치

- 하드 디스크는 데이터를 읽고 쓰기 위해 움직이는 헤드를 사용하는 구조

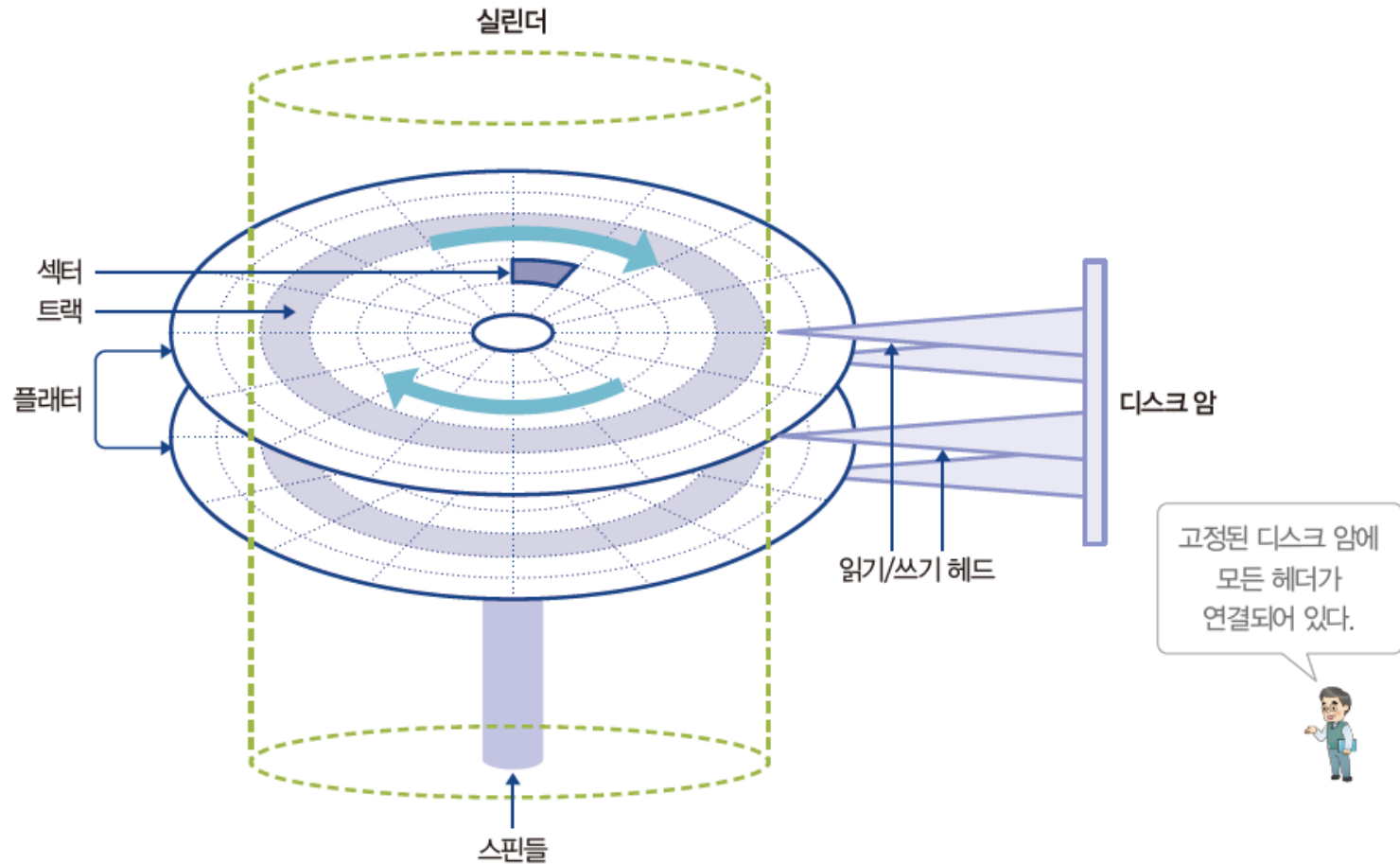


그림 12-1 하드디스크의 구조

01 저장장치와 디스크 스케줄링

1. 자기 디스크 기반 저장장치

- 섹터: 하드디스크의 최소 저장 단위
 - 블록이 여러 개의 섹터에 나뉘어 저장됨
- 트랙: 플래터에서 회전축 중심으로 데이터가 기록되는 동심원
- 실린더: 여러 개의 플래터에서 같은 위치에 있는 트랙의 개념적인 집합

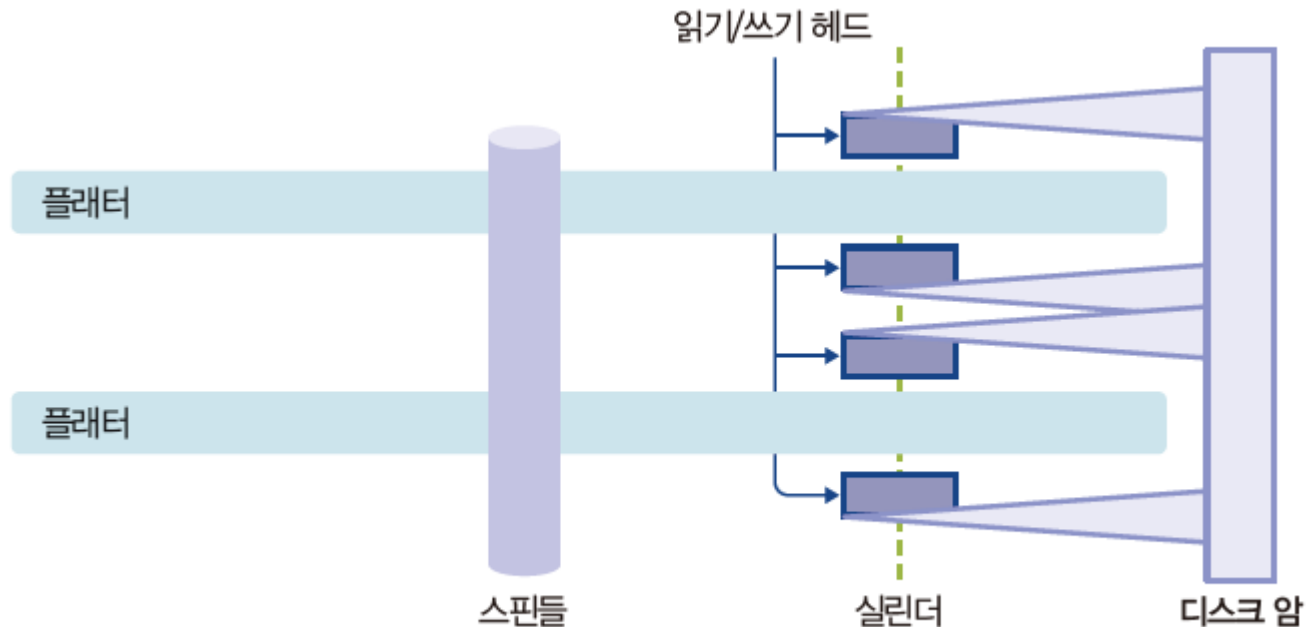


그림 12-2 하드디스크의 헤드와 플래터

01 저장장치와 디스크 스케줄링

2. 반도체 기반 저장장치

- SSD는 비휘발성 메모리인 플래시 메모리를 이용하여 데이터를 저장하는 저장장치.
- 입출력 속도가 빠름
- NVMe는 하드디스크와 다른 독자 인터페이스
↔ SATA 인터페이스



그림 12-3 SATA 인터페이스 SSD와 NVMe 인터페이스 SSD

01 저장장치와 디스크 스케줄링

3. 광 디스크 저장장치

- 레이저를 이용한 저장장치: CD, DVD, 블루레이 디스크
- 고용량 게임 배포에 사용됨



그림 12-4 CD에서 데이터를 읽는 방식

01 저장장치와 디스크 스케줄링

3. 광 디스크 저장장치

- 하드디스크는 각속도 일정 방식, 일정한 시간 동안 이동한 각도가 같음.
- CD는 선속도 일정 방식, 어느 트랙에서나 단위 시간당 디스크의 이동 거리가 같음.

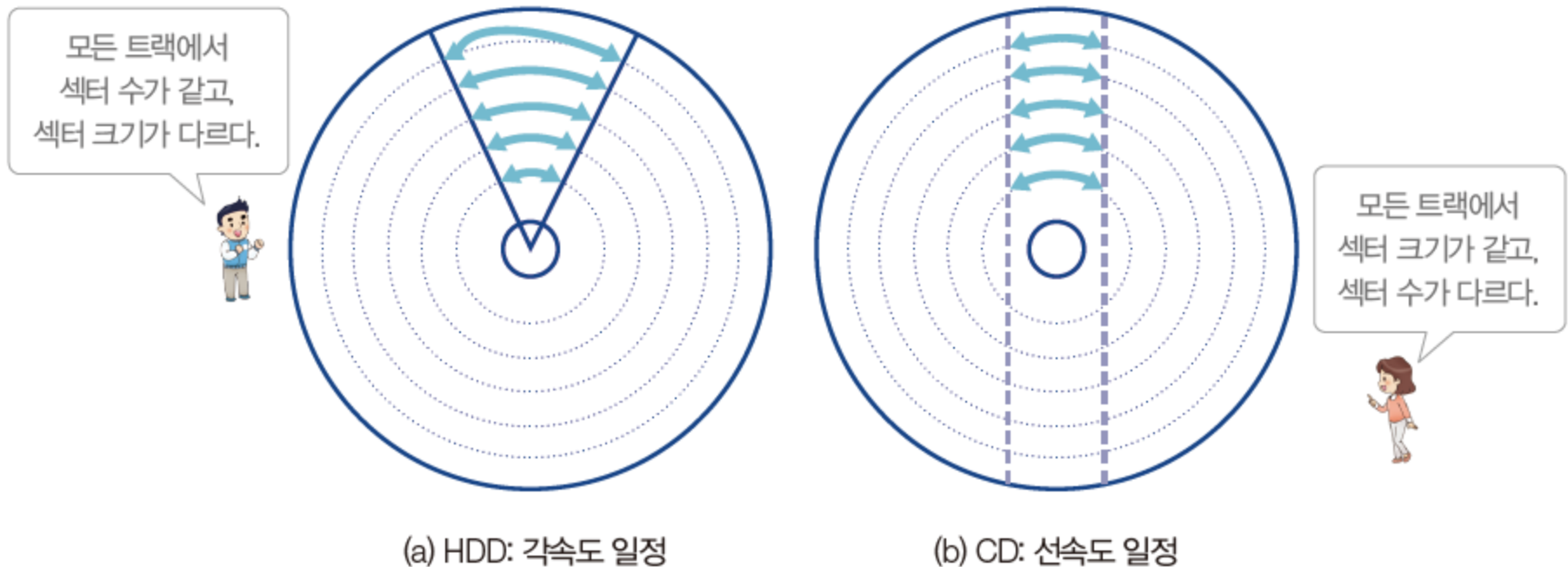


그림 12-5 디스크 장치의 회전

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆데이터 전송 시간과 디스크 스케줄링의 필요성

데이터 전송 시간 = 탐색 시간 + 회전 지연 시간 + 전송 시간

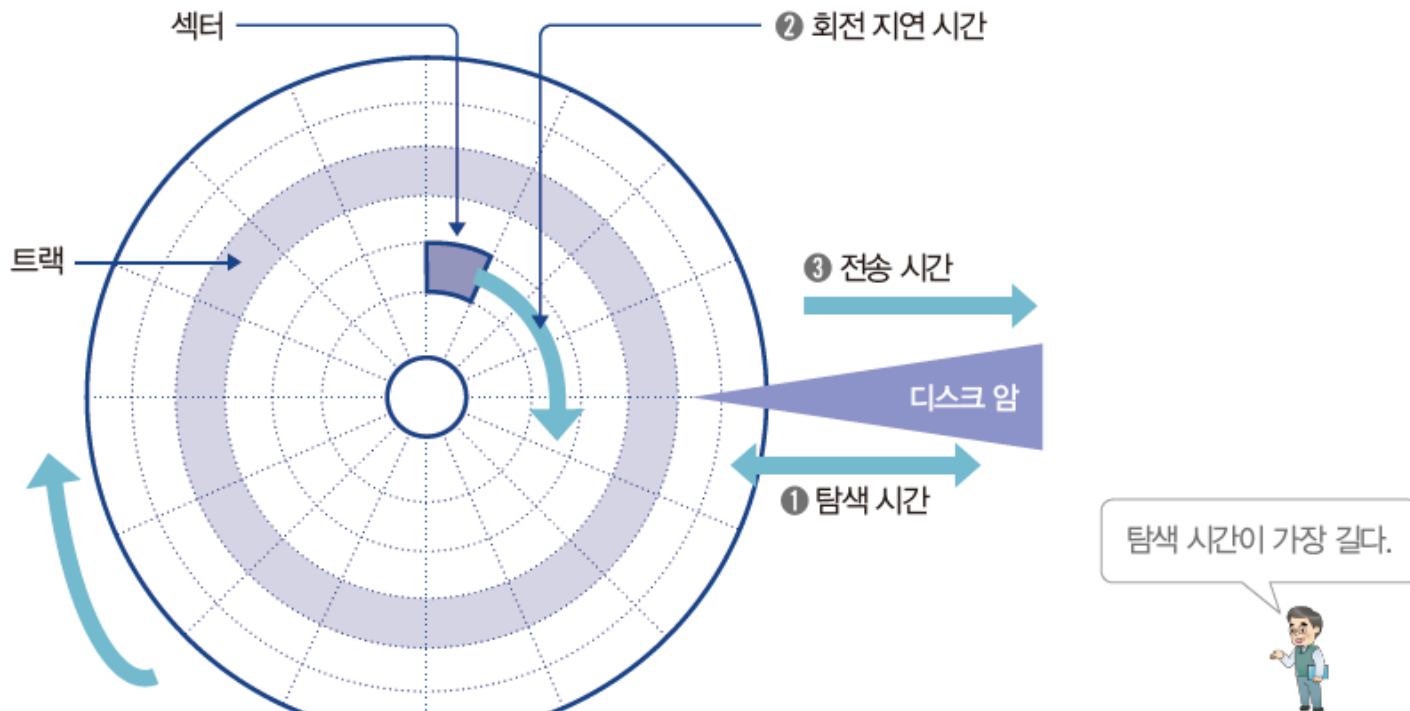


그림 12-6 디스크를 사용하는 저장장치의 데이터 전송 과정

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆데이터 전송 시간과 디스크 스케줄링의 필요성

- 이후에 다양한 디스크 스케줄링을 소개하는데,
아래 표의 순서대로 트랙 접근 요청을 받았다고 가정함.

표 12-1 트랙 접근 순서

순번	1	2	3	4	5	6	7	8	9
트랙 번호	15	8	17	11	3	23	19	14	20

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ FCFS 디스크 스케줄링

- 요청이 들어온 트랙 순서대로 서비스
- 가장 단순한 디스크 스케줄링 방식

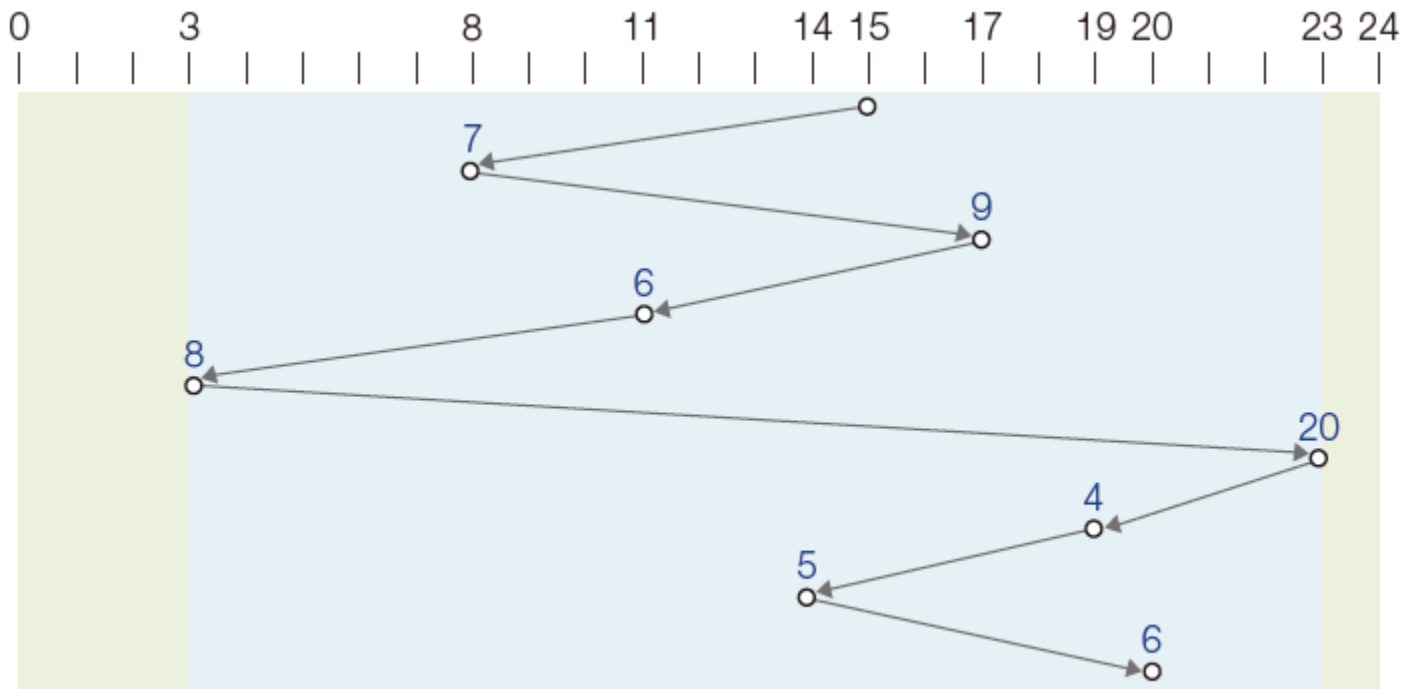


그림 12-7 FCFS 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ SSTF 디스크 스케줄링

- 현재 헤드의 위치에서 가장 가까운 트랙부터 서비스.

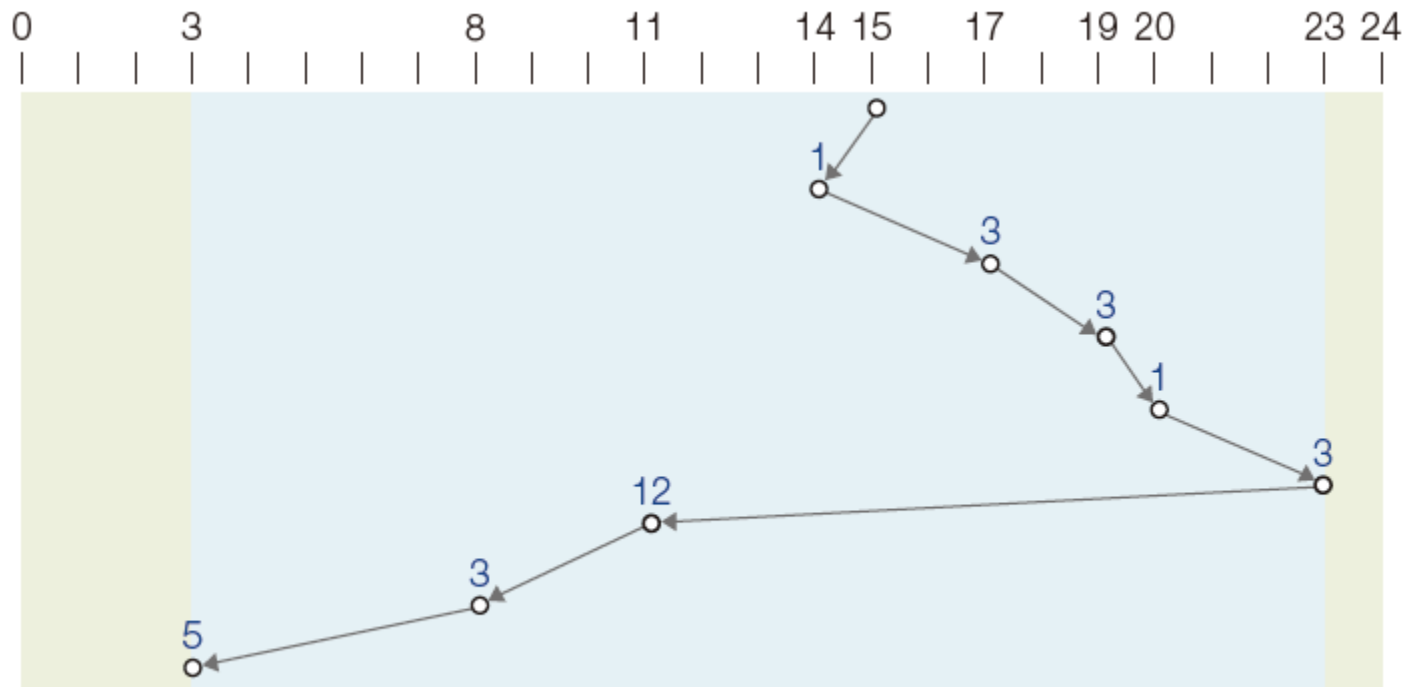


그림 12-8 SSTF 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ 블록 SSTF 디스크 스케줄링

- SSTF 디스크 스케줄링의 공평성 위배를 일부 해결
- 트랙 요청을 아래 표와 같이 일정한 크기의 블록으로 묶고, 블록 안에서만 순서 변경

표 12-2 블록 SSTF 디스크 스케줄링의 서비스 순서 변경

블록	1			2			3		
순번	1	2	3	4	5	6	7	8	9
트랙 번호	15	8	17	11	3	23	19	14	20
서비스	15	17	8	11	3	23	20	19	14

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ 블록 SSTF 디스크 스케줄링

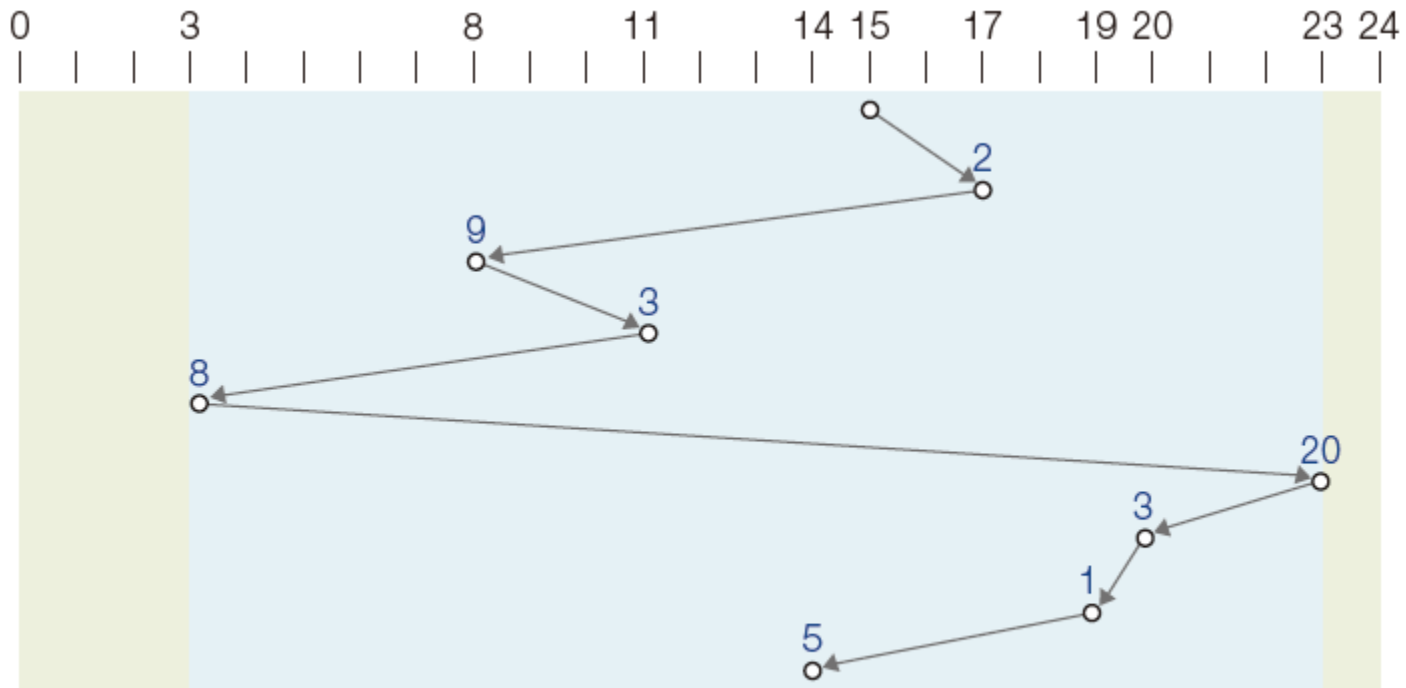


그림 12-9 블록 SSTF 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ SCAN 디스크 스케줄링

- 헤드가 스캐너처럼 한 방향으로만 움직임.
- SSTF 디스크 스케줄링의 공정성 위배 문제를 완화

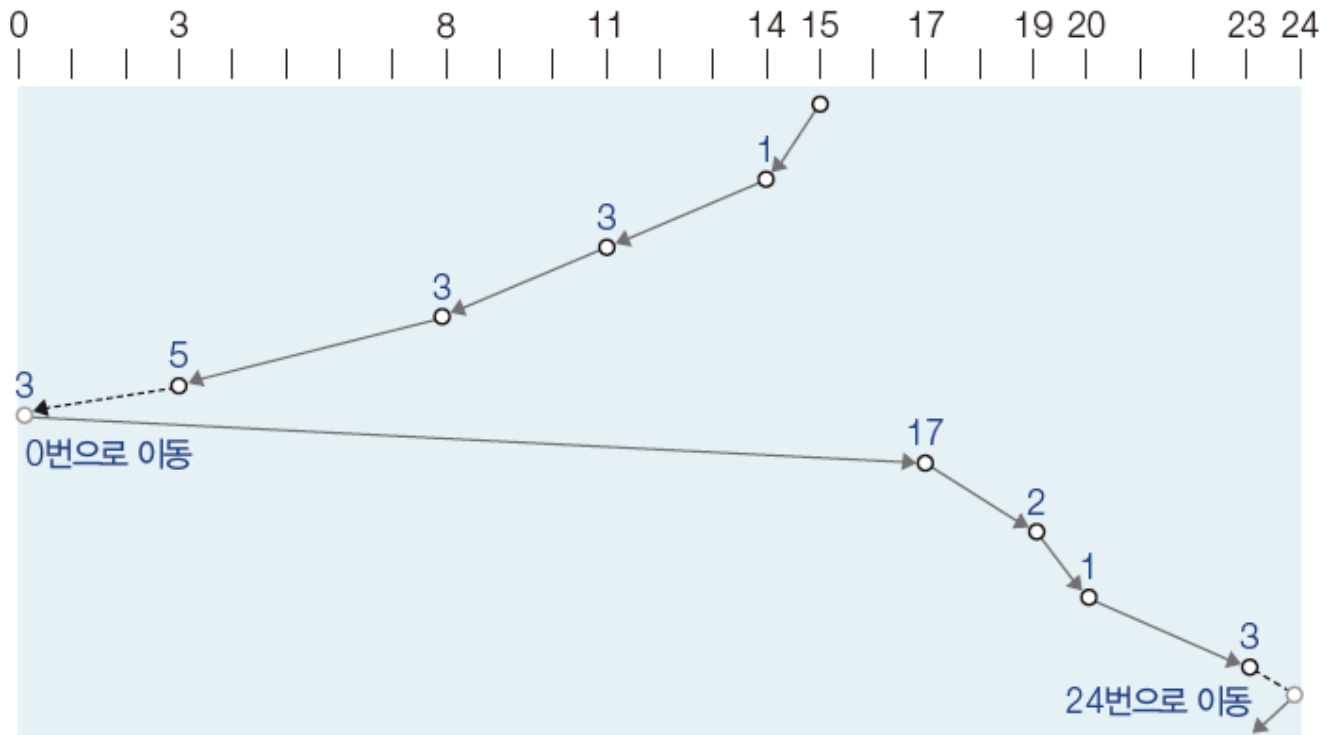


그림 12-10 SCAN 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ C-SCAN 디스크 스케줄링

- 한쪽 방향으로만 요청받은 트랙을 서비스하고, 반대로 돌아올 때는 작업 없이 이동
- SCAN 스케줄링의 공정성 문제(바깥쪽 트랙보다 중간에 있는 트랙 자주 방문)를 완화

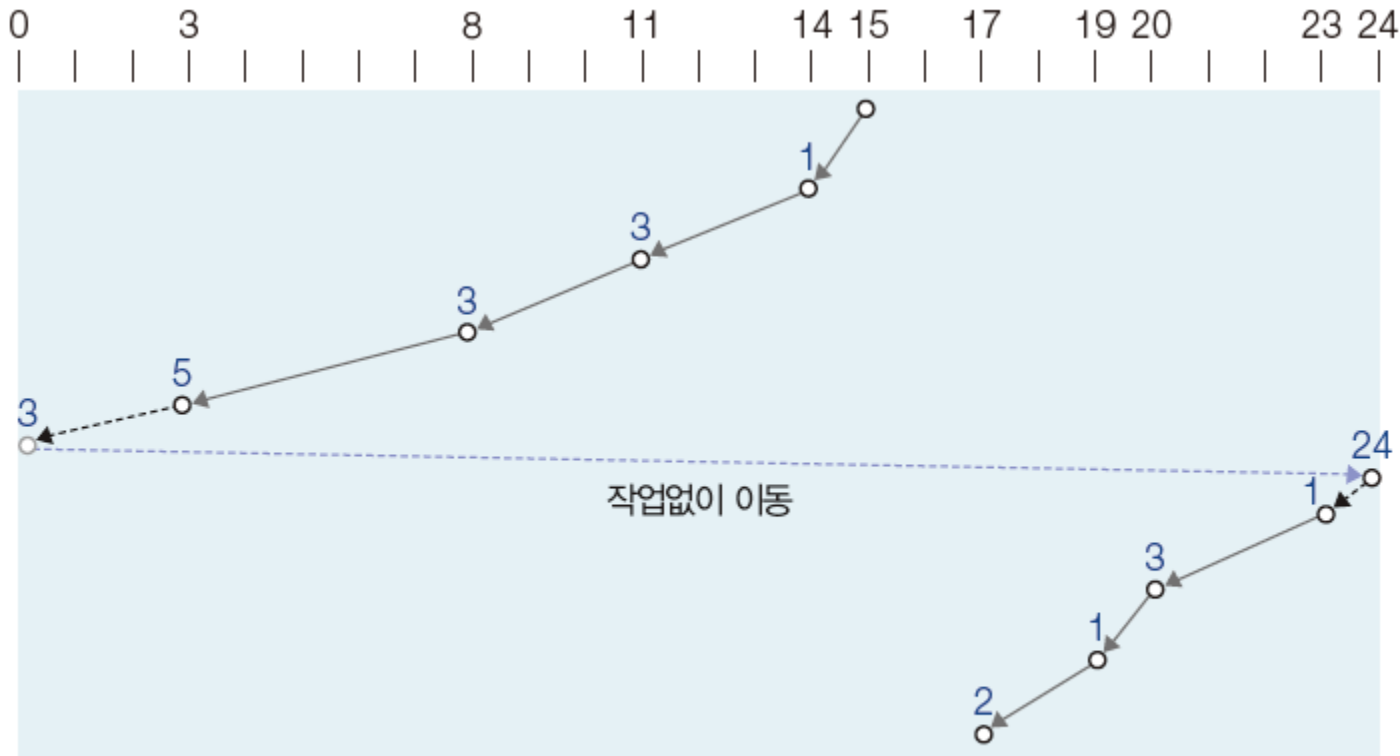


그림 12-11 C-SCAN 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ LOOK 디스크 스케줄링

- SCAN 디스크 스케줄링의 불필요한 부분을 제거하여 효율을 높임

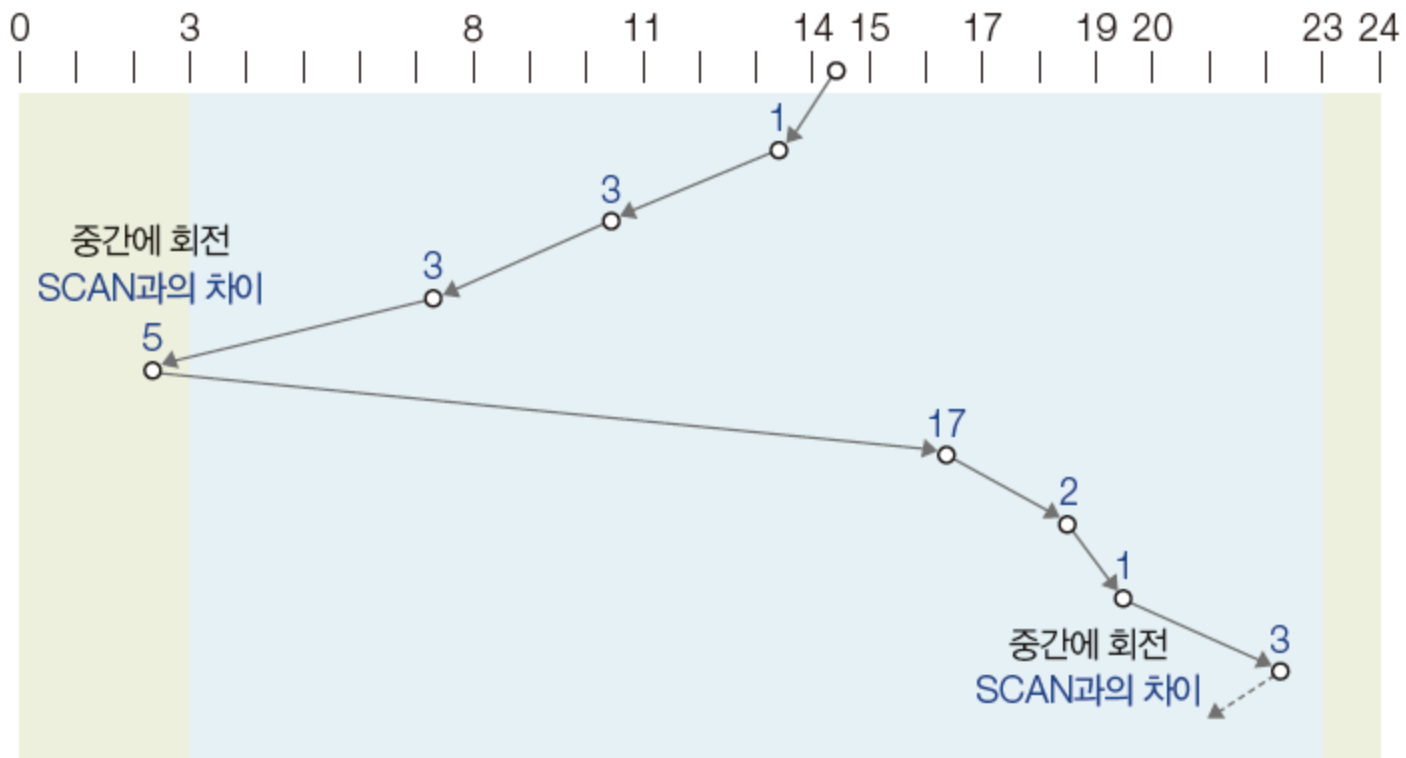


그림 12-12 LOOK 디스크 스케줄링

01 저장장치와 디스크 스케줄링

4. 디스크 스케줄링 알고리즘

◆ C-LOOK 디스크 스케줄링

- C-SCAN 디스크 스케줄링에 LOOK을 적용한 기법.

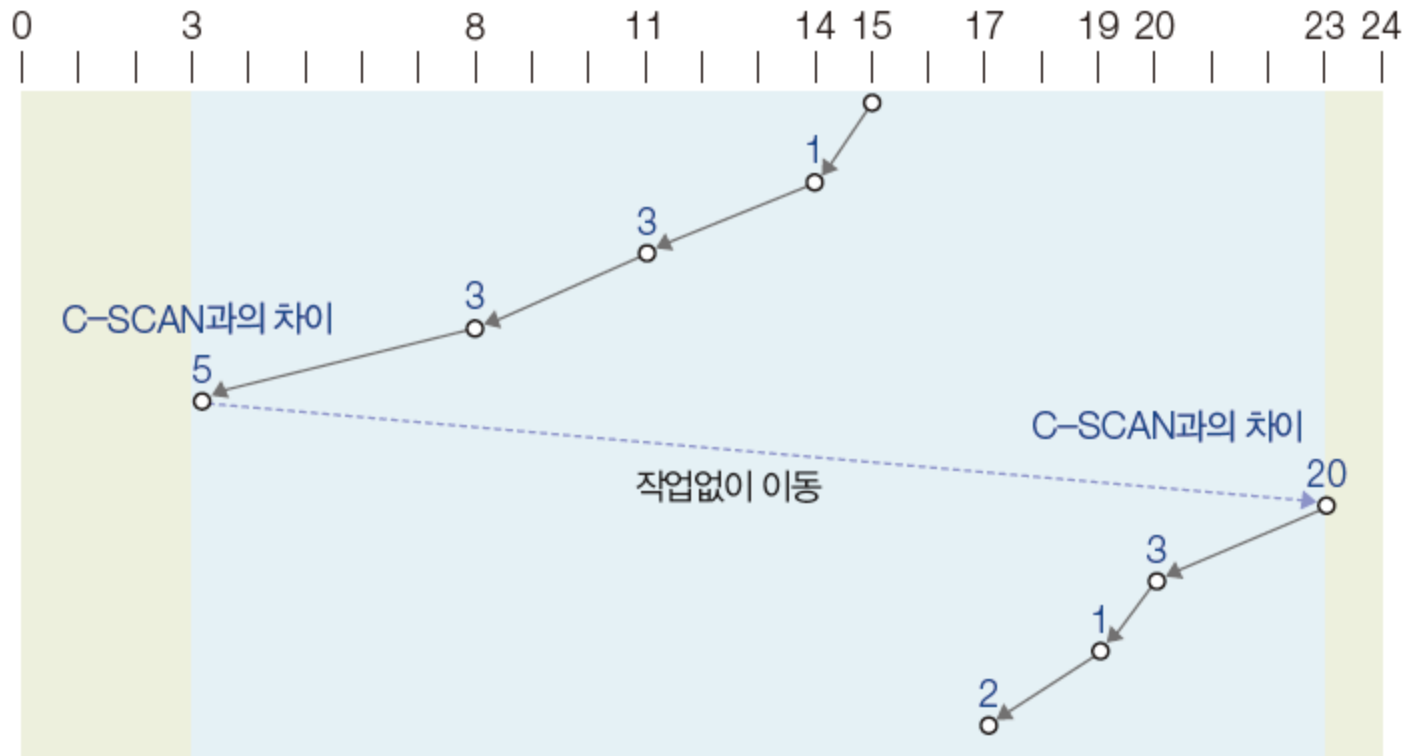


그림 12-13 C-LOOK 디스크 스케줄링의 동작

02

RAID

02 RAID

1. 중복 배열 디스크

- RAID(Redundant Array of Independent Disks)
- 데이터 보호와 복구를 자동화하여 디스크 장애 시에도 시스템을 안정적으로 유지하는 저장장치 시스템.
- 여러 개의 디스크를 논리적으로 묶어 사용

02 RAID

2. RAID 레벨별 특징

◆ RAID 0: 스트라이핑

- 여러 개의 디스크에 나누어 동시에 저장함으로써 입출력 속도를 높임

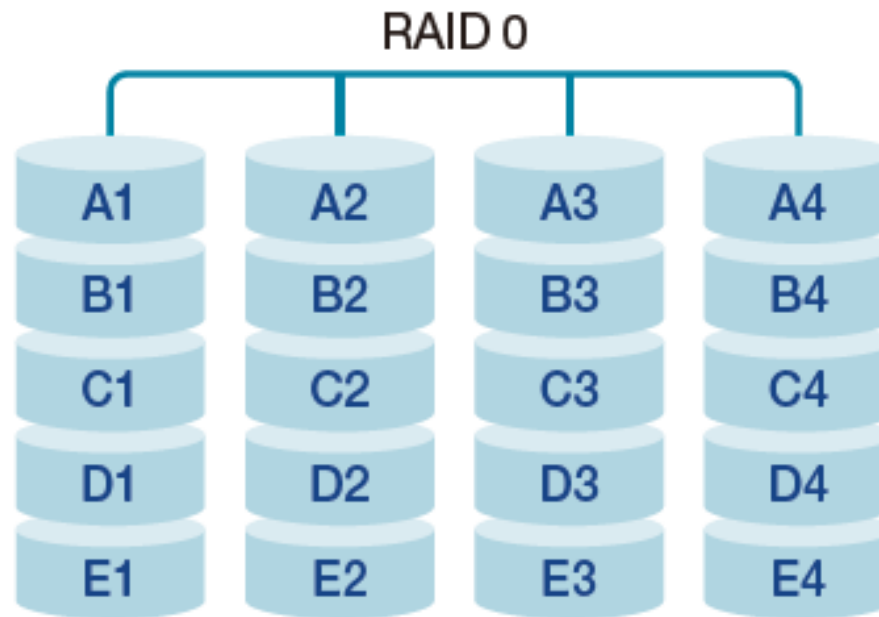


그림 12-14 RAID 0

02 RAID

2. RAID 레벨별 특징

◆ RAID 1: 미러링

- 2개의 디스크에 같은 데이터를 저장하여, 디스크 1개는 백업 디스크로 활용
 - 순수한 백업 시스템.
- 한쪽 디스크의 데이터의 복사본이 다른 디스크에 만들어지는 **미러링** 방식으로 동작.

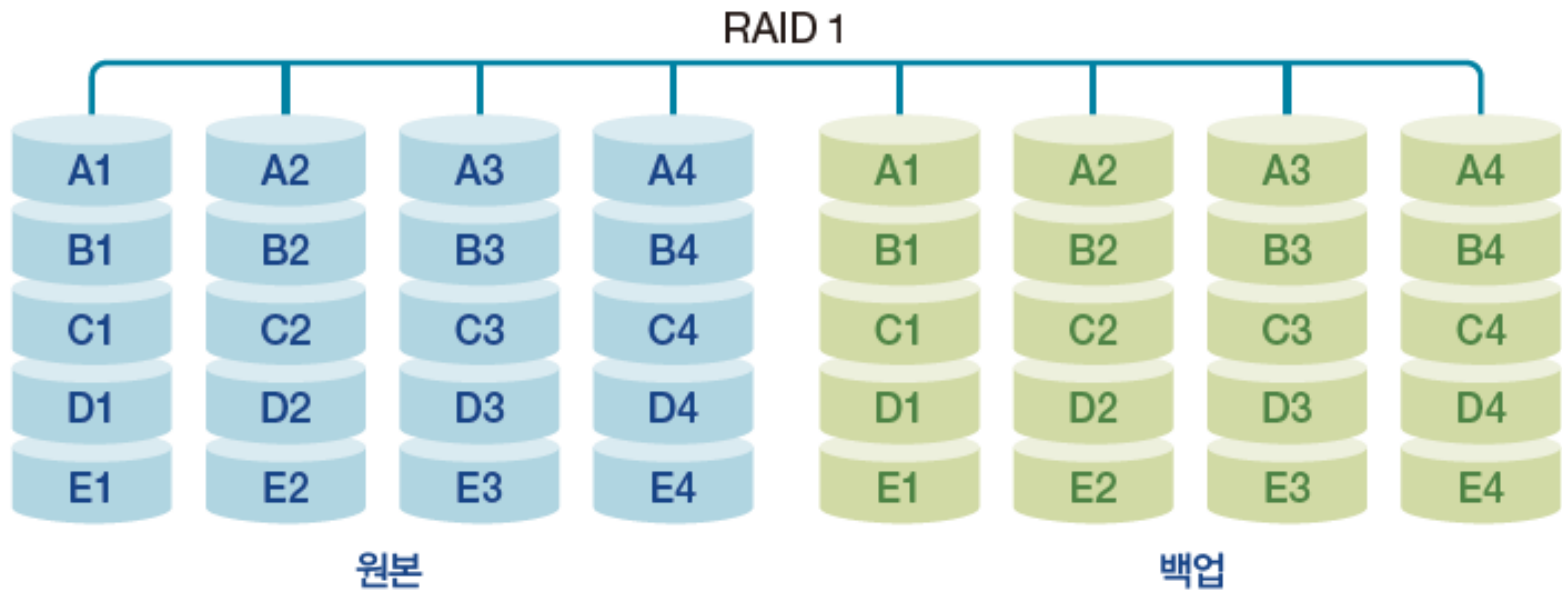


그림 12-15 RAID 1

02 RAID

2. RAID 레벨별 특징

◆ RAID 2

- 디스크에 저장된 데이터에 오류가 발생했을 때 오류 정정 코드를 이용해 복구

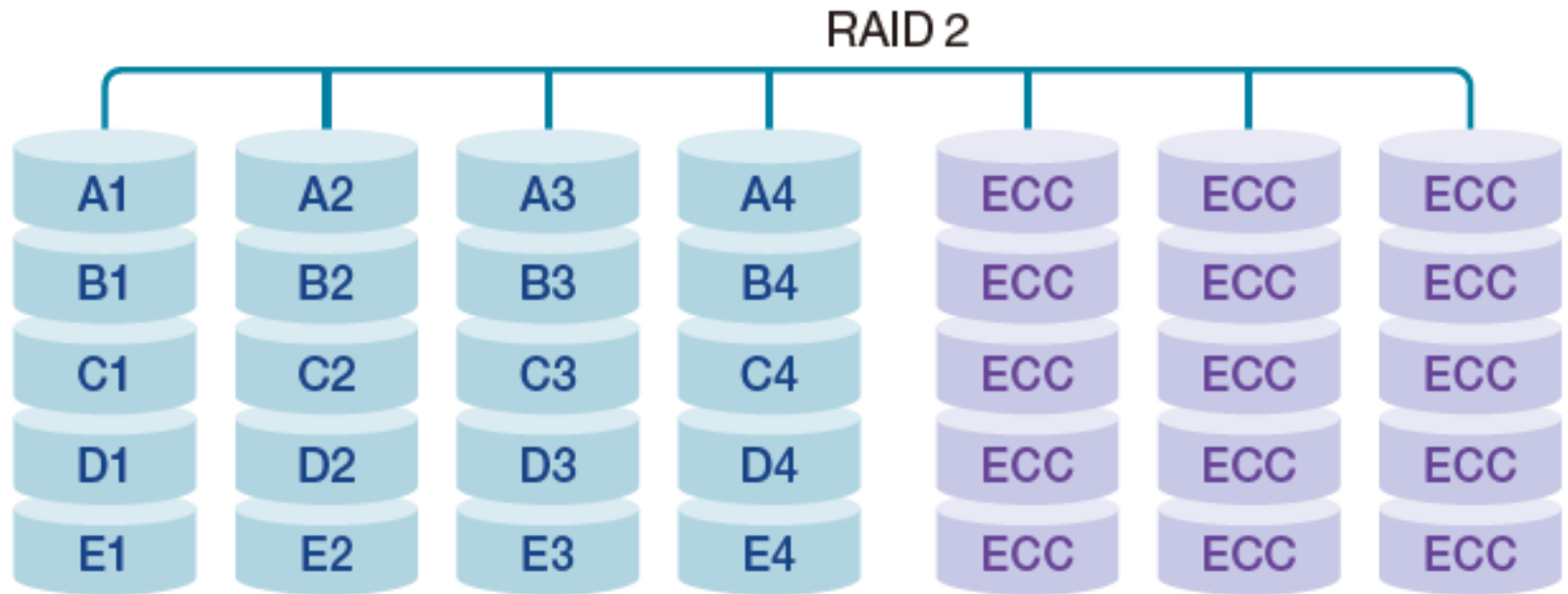


그림 12-16 RAID 2

02 RAID

2. RAID 레벨별 특징

◆ RAID 3과 RAID 4: 패리티 기반 방식

- 오류 검출 코드인 패리티 비트를 사용하여 데이터를 복구
- RAID에서는 패리티 비트로도 오류를 복구 가능.



그림 12-17 패리티 비트를 이용한 오류 복구

02 RAID

2. RAID 레벨별 특징

◆ RAID 3과 RAID 4: 패리티 기반 방식

- RAID 3에서는 데이터를 섹터 단위로 여러 디스크에 나누어 저장
 - 추가 디스크 양이 적지만, N-way 패리티 비트 구성에 계산량이 많음
 - 모든 디스크가 동시에 동작해야 패리티 비트 구성 가능

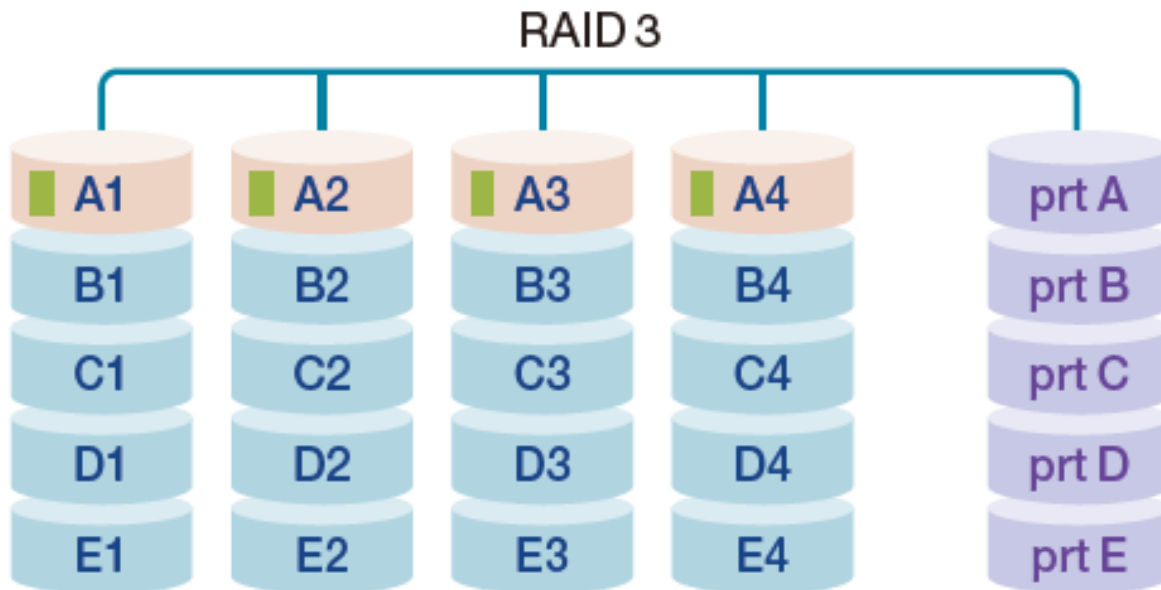


그림 12-18 RAID 3

02 RAID

2. RAID 레벨별 특징

◆ RAID 3과 RAID 4: 패리티 기반 방식

- RAID 4에서는 데이터를 하나의 디스크에 블록 단위로 저장하고 패리티 비트를 블록과 연결
 - 입출력 시마다 패리티 비트 디스크에 병목 현상

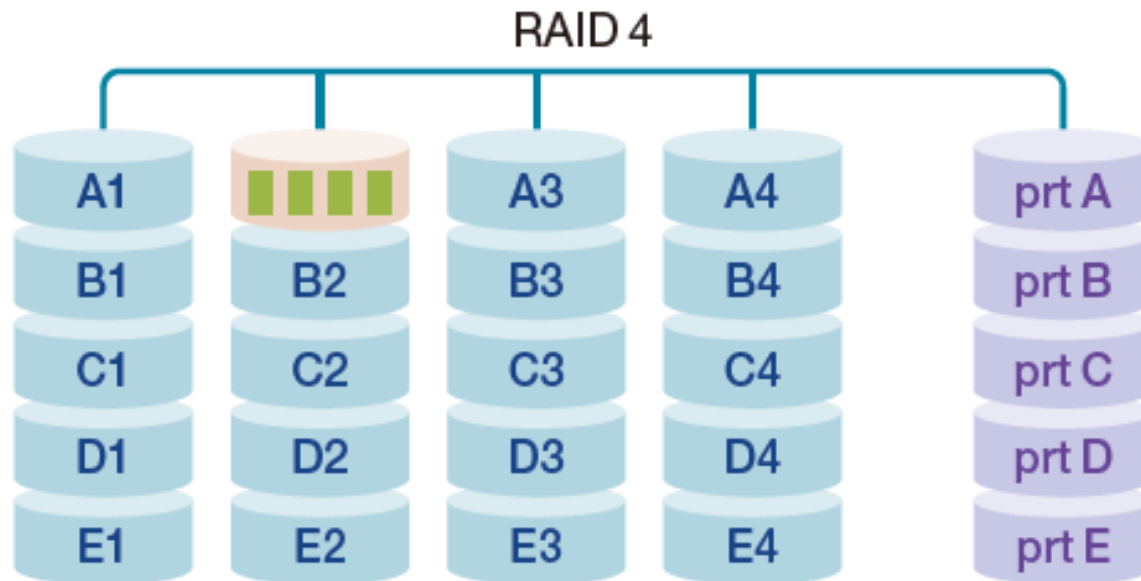


그림 12-19 RAID 4

02 RAID

2. RAID 레벨별 특징

◆ RAID 5와 RAID 6: 분산 패리티 기반 고신뢰성 방식

- RAID 5는 패리티 비트를 사용하여 데이터를 복구하지만 병목 현상을 해결한 방식.
 - RAID 4에서 패리티 비트 디스크가 고장 시 복구가 어렵다는 문제를 해결.
 - 패리티 비트를 해당 데이터가 없는 디스크에 분산하여 구성.

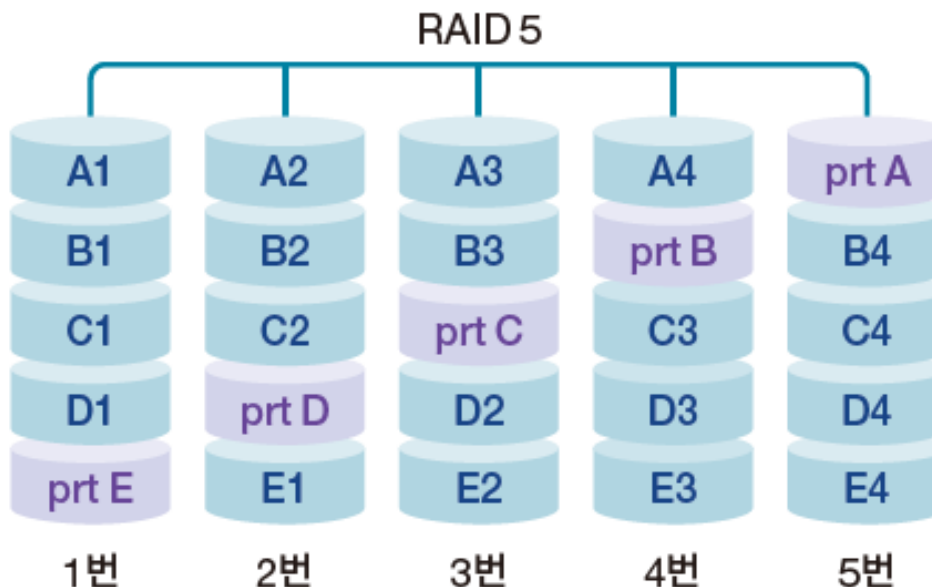


그림 12-20 RAID 5

02 RAID

2. RAID 레벨별 특징

◆ RAID 5와 RAID 6: 분산 패리티 기반 고신뢰성 방식

- RAID 6은 패리티 비트를 2개로 구성하고 서로 다른 디스크에 분산

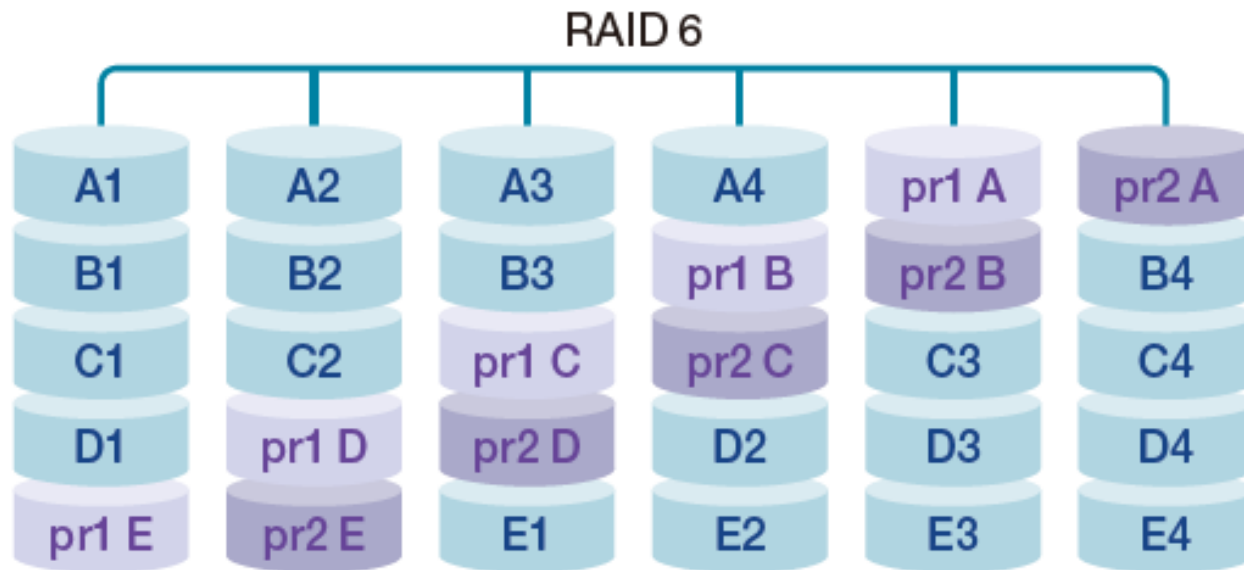


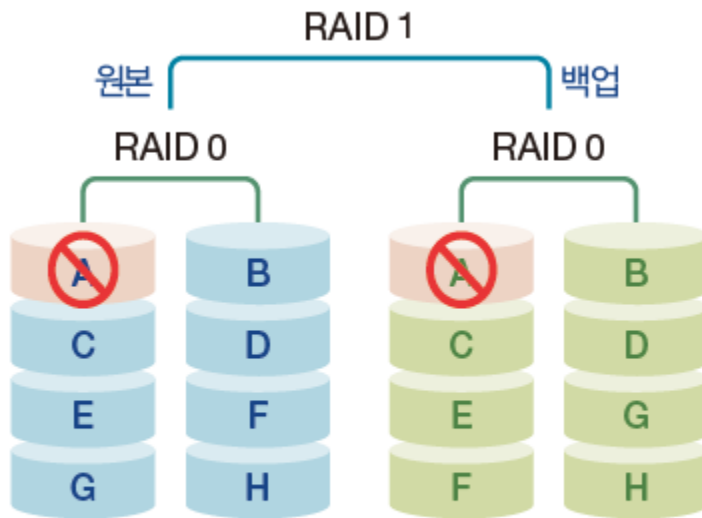
그림 12-21 RAID 6

02 RAID

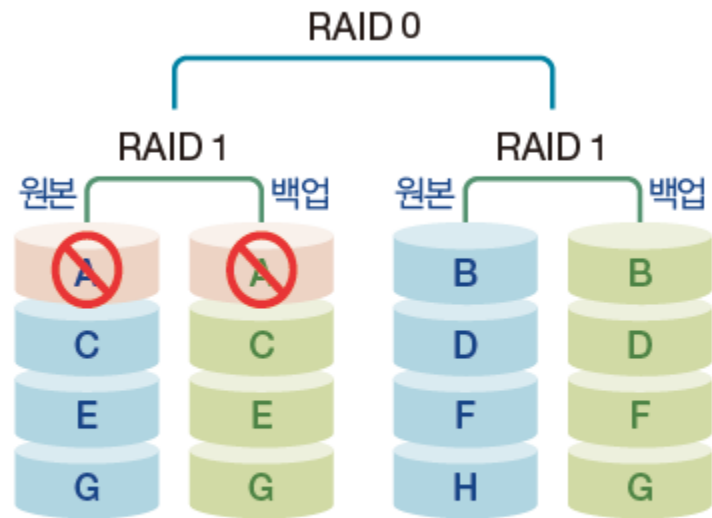
3. 복합 RAID

◆ RAID 0+1과 RAID 10

- RAID 0+1은 RAID 0으로 구성한 디스크 2개와 2개를 RAID 1로 결합한 것.
- RAID 10은 RAID 1로 구성한 디스크 2개와 2개를 RAID 0으로 결합한 것.
 - RAID 10은 장애 발생 시 일부 디스크만 중단해도 복구 가능



(a) RAID 0+1



(b) RAID 10

그림 12-22 RAID 0+1과 RAID 10

02 RAID

3. 복합 RAID

◆ RAID 50과 RAID 60

- 원본과 백업을 RAID 0으로 묶어 성능을 높임
- RAID 50은 RAID 5로 묶은 쌍을 다시 RAID 0으로 묶어 사용.
- RAID 60은 RAID 6으로 묶은 쌍을 다시 RAID 0으로 묶어 사용.

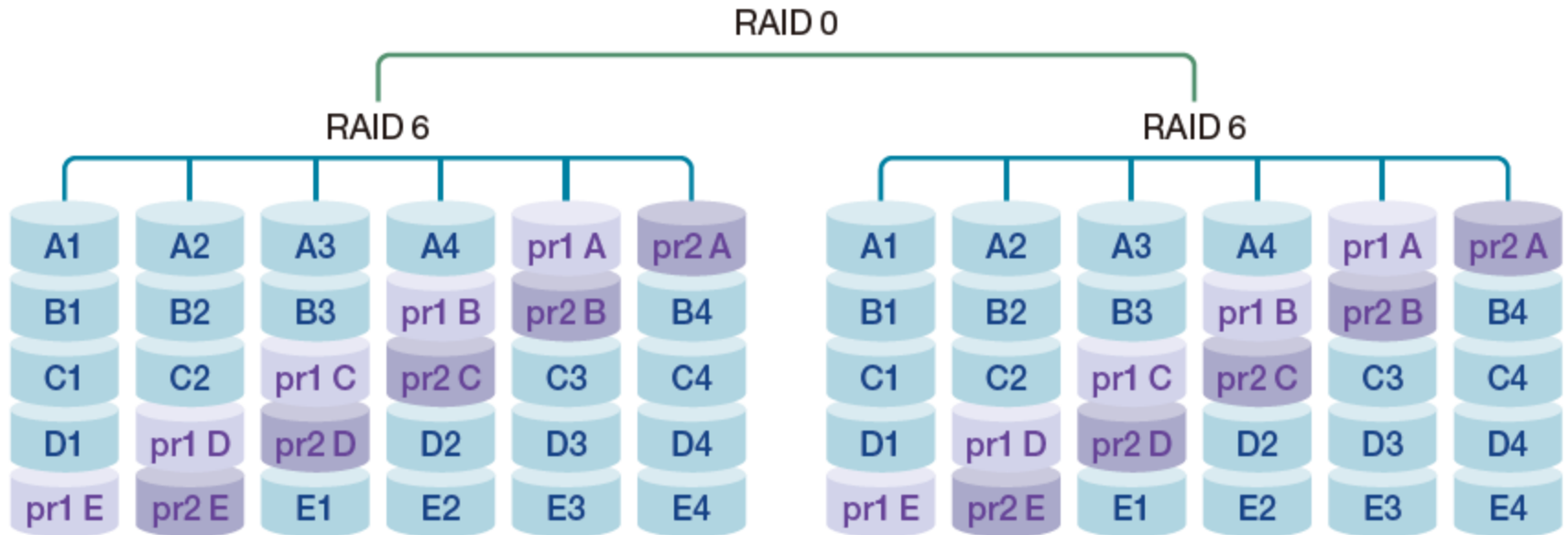


그림 12-23 RAID 60의 구조

03

입출력 시스템

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆저속장치와 고속장치

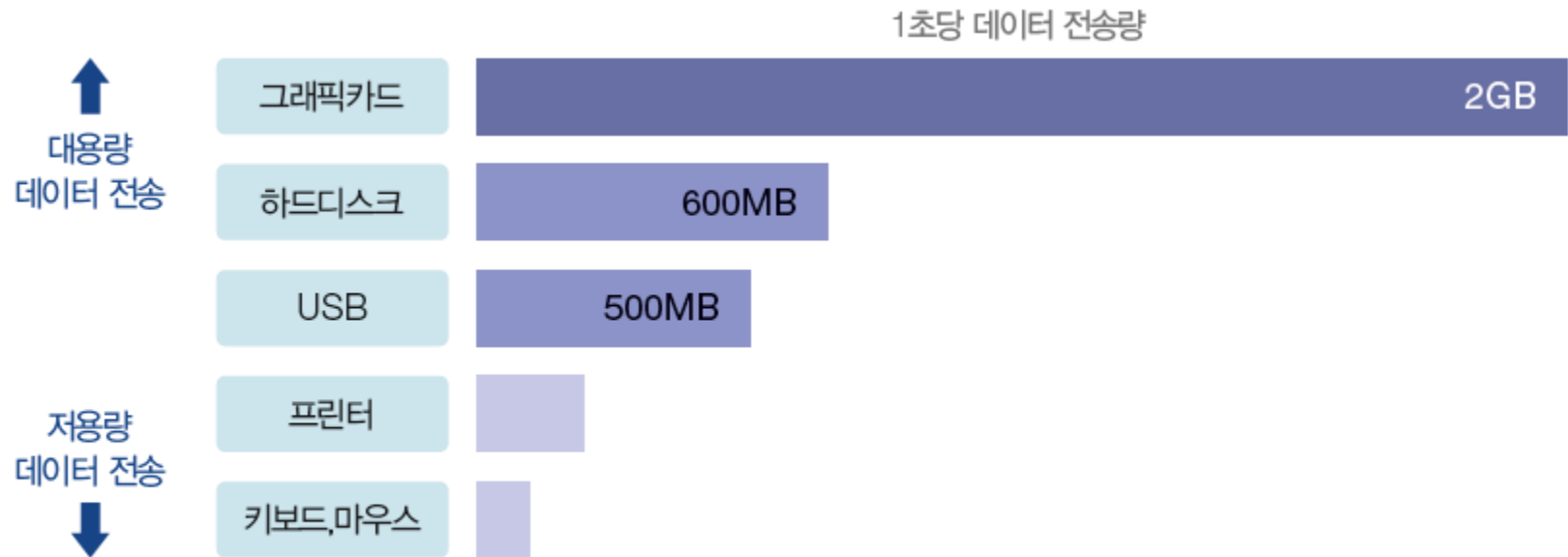


그림 12-24 주변장치의 데이터 전송 속도

03 입출력 시스템

1. 입출력 장치와 버스 구성

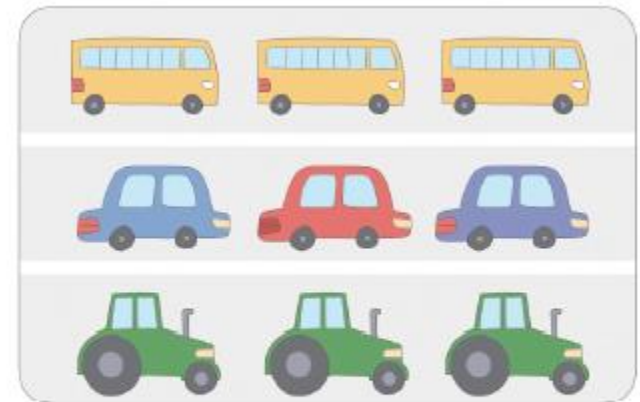
◆저속장치와 고속장치

- 채널: 데이터가 지나다니는 하나의 통로



(a) 모든 장치가 채널을 공유

채널을 공유하면
저속 장치 때문에
고속 장치가 느려진다.



(b) 장치 속도별로 채널을 분리

그림 12-25 채널 공유와 채널 분리

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆ 입출력 버스 구조

- 입출력 제어기가 입출력 장치를 관리.
 - 메인 버스와 입출력 버스를 나누어 관리

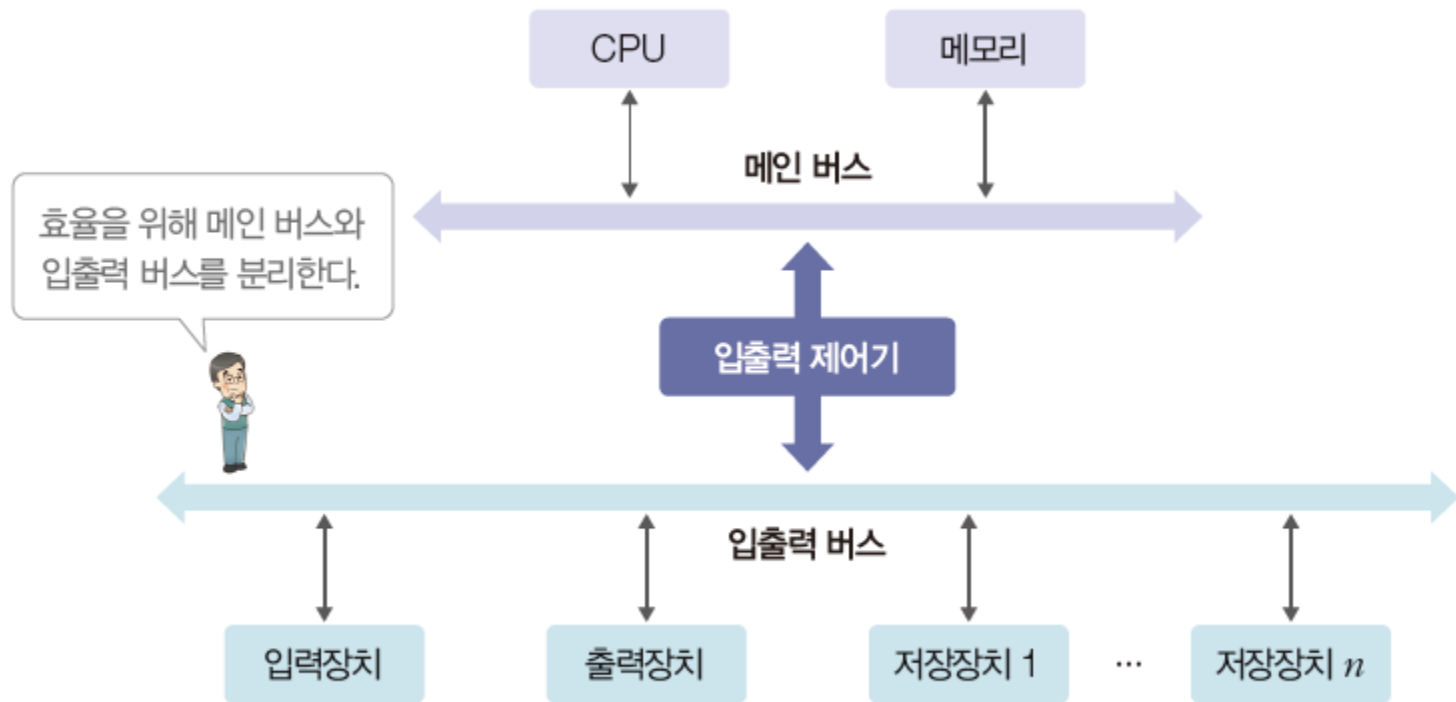


그림 12-26 입출력 제어기가 있을 때 입출력 버스의 구조

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆ 입출력 버스 분리

- 고속 주변장치가 저속 주변장치와 입출력 버스를 공유하면 입출력 속도가 저하되므로 고속 입출력 버스와 저속 입출력 버스로 분리하여 운영.

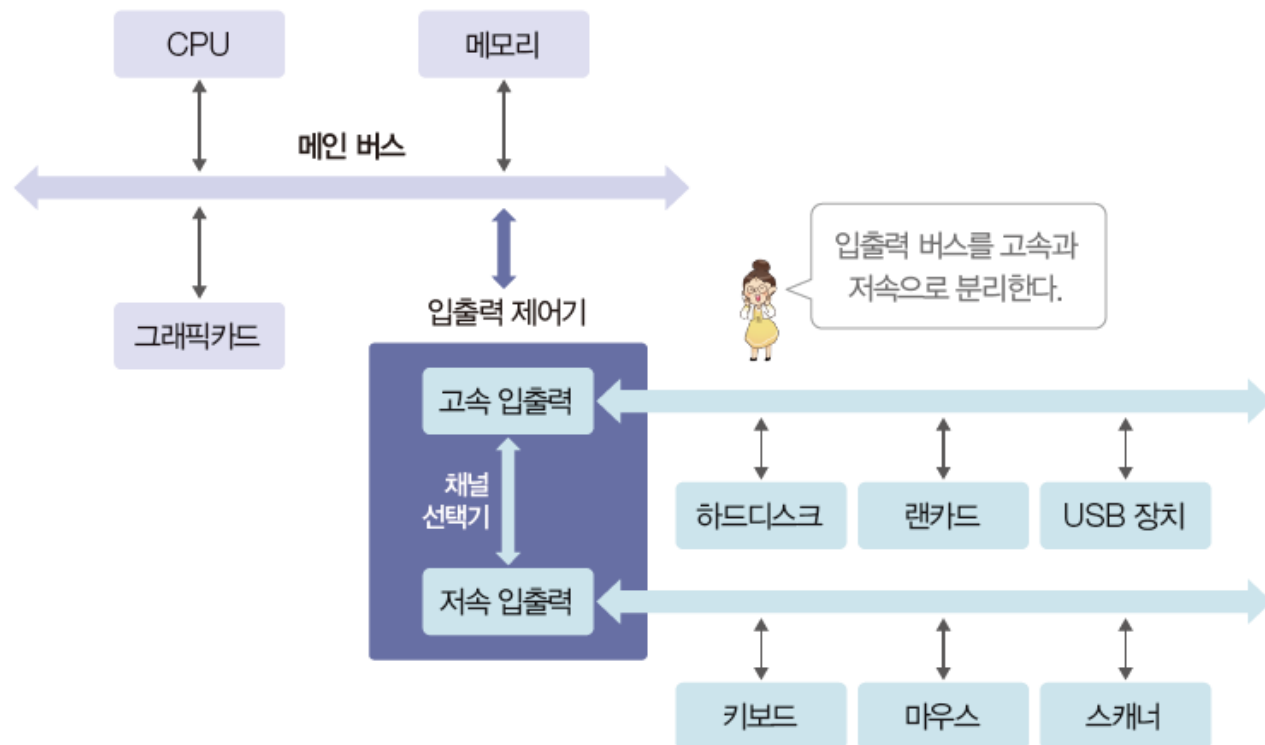


그림 12-27 입출력 버스의 분리

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆그래픽카드

- GPU는 실수 연산(그래픽 관련 연산)을 빠르게 처리하도록 설계됨
- 최근 딥러닝 모델이나 가상화폐 채굴 등에서 활용.
 - 대용량의 데이터를 빠르게 처리해야 하는 분야

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆ 포트

- 장치 규격에 맞춰 데이터를 주고받는 접속 단자.
- 컴퓨터와 주변장치를 물리적으로 연결.

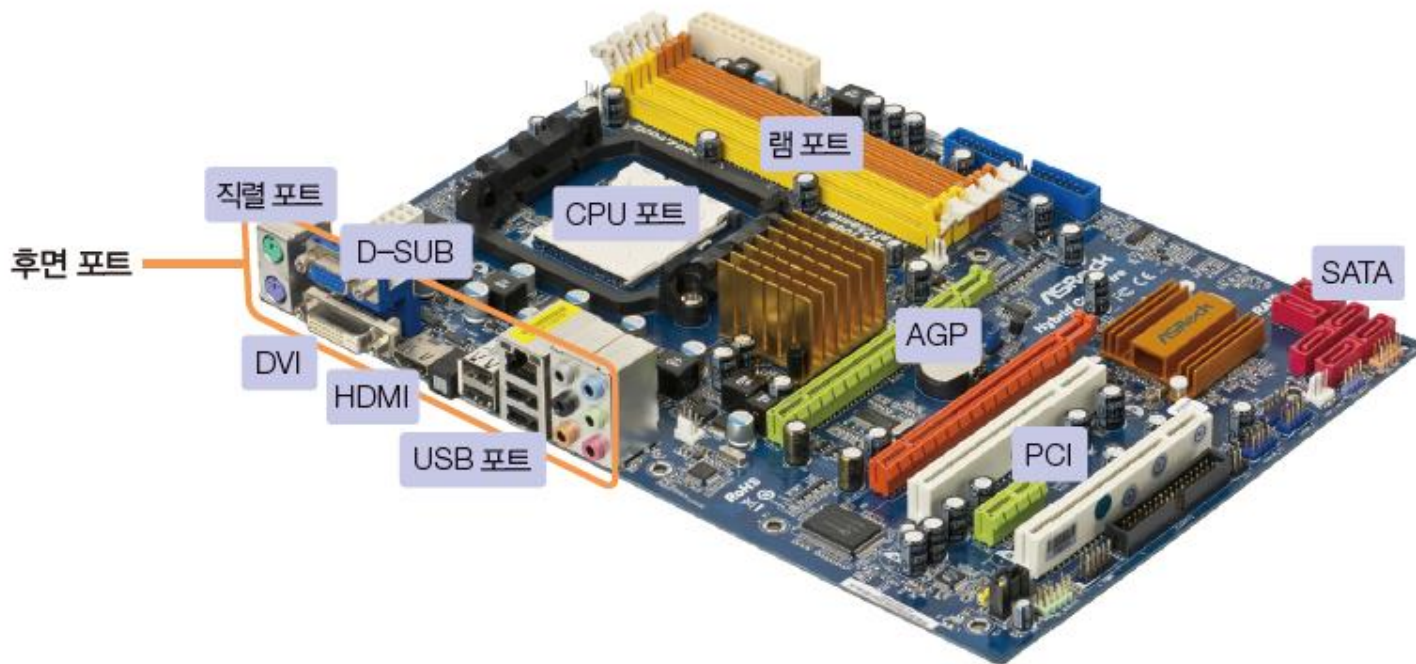


그림 12-28 메인보드의 포트

03 입출력 시스템

1. 입출력 장치와 버스 구성

◆ 포트



USB



USB-C



D-SUB



DVI



HDMI

그림 12-29 케이블 단자

03 입출력 시스템

2. 입출력 제어 방식

◆ 직접 메모리 접근(DMA Direct Memory Access)

- 입출력 장치가 CPU의 허락 없이도 메모리와 데이터를 주고 받을 수 있는 권한.
- DMA 제어기가 CPU를 대신하여 메모리와 입출력 장치 간의 데이터 전송을 조정.
 - CPU의 개입 없이 독립적으로 작업 수행.

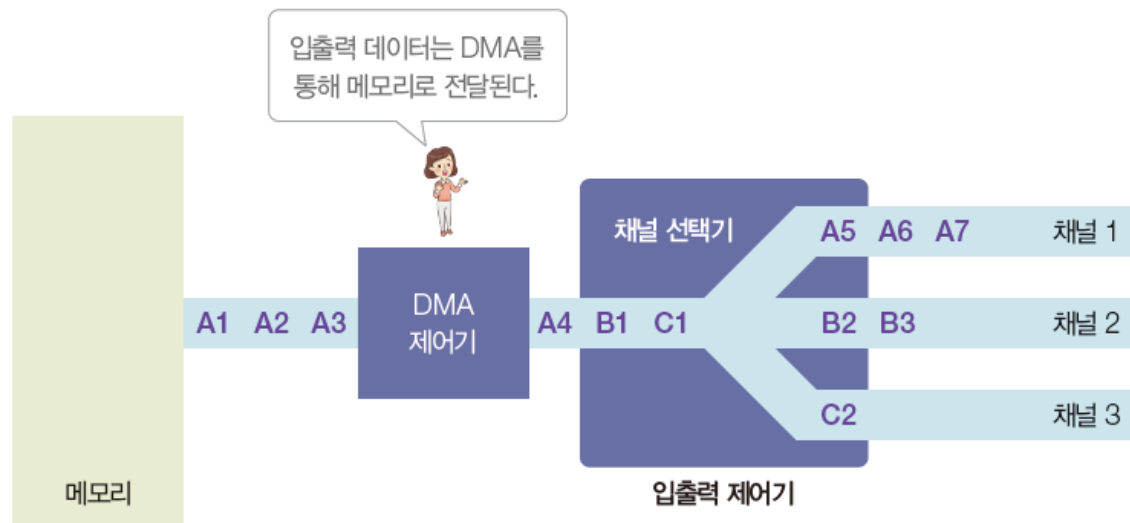


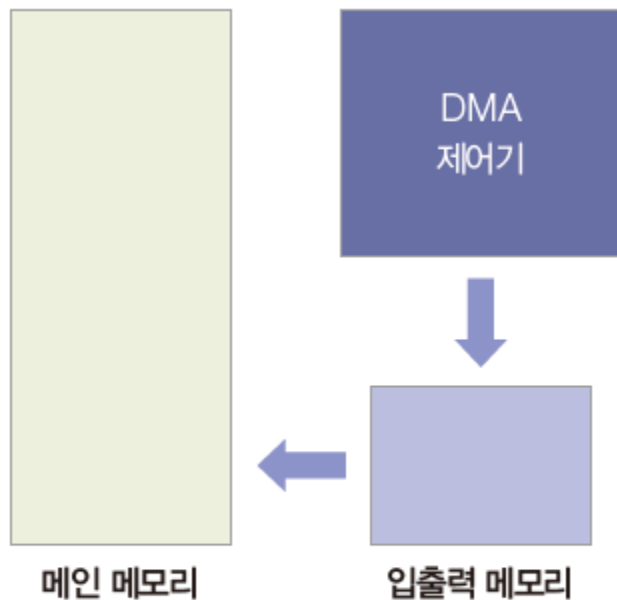
그림 12-30 입출력 제어기

03 입출력 시스템

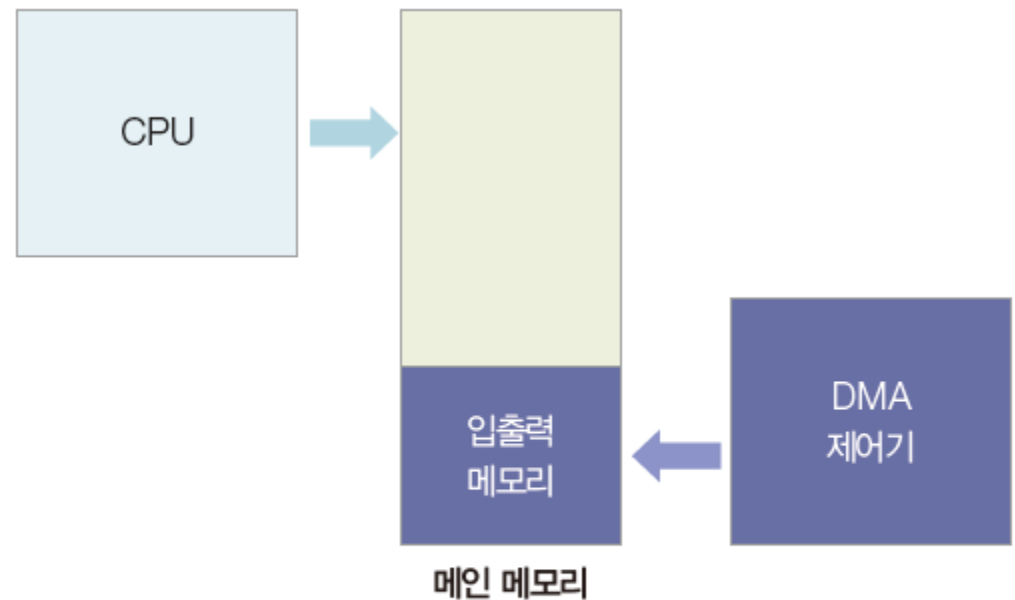
2. 입출력 제어 방식

◆메모리 맵 입출력

- CPU와 DMA의 충돌을 방지하기 위해 메인 메모리의 일부를 DMA 제어기에 할당



(a) 입출력 메모리 사용



(b) 메모리 맵 입출력

그림 12-31 메모리 공간 분할

03 입출력 시스템

2. 입출력 제어 방식

◆ 인터럽트

- 주변장치의 입출력 요구나 하드웨어의 이상 현상을 CPU에 알려주는 신호.
- CPU가 인터럽트를 받았을 때 어떤 장치로부터 받았는지 확인할 수 있도록 각 장치에 인터럽트 번호 부여.

표 12-3 인터럽트 번호 할당

종류	인터럽트 번호	설명
내부 인터럽트	0~31번	CPU 내부에서 발생하며, CPU가 오류를 직접 처리함
외부 인터럽트	32~47번	하드웨어 장치의 요청으로 발생하며, IRQ 번호에 매핑됨
소프트웨어 시그널	128번 이상	사용자나 커널에 의해 발생하는 소프트웨어 시그널

- 예외 인터럽트
- 인터럽트 벡터

03 입출력 시스템

2. 입출력 제어 방식

◆ 인터럽트

- 인터럽트 2번과 인터럽트 5번이 동시에 발생했다면?

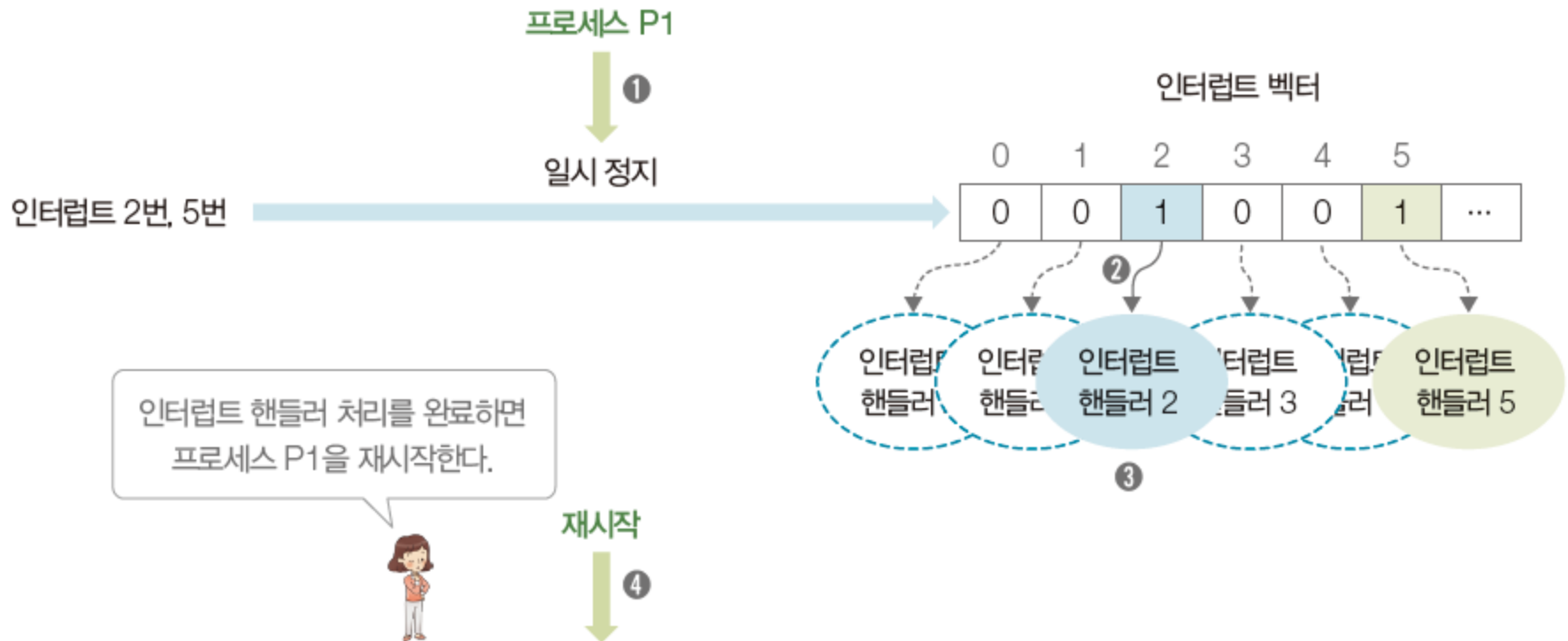


그림 12-32 인터럽트 처리 과정

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 인터럽트의 종류

- 동기적 인터럽트: 명령어 실행 결과로 발생하는 인터럽트.
 - 예외라고도 함
- 비동기적 인터럽트: 하드웨어 장치가 CPU의 현재 명령 실행과 관계없이 발생시킴
 - 외부 인터럽트

표 12-4 인터럽트 종류 비교

구분	동기적 인터럽트	비동기적 인터럽트
정의	명령어 실행 결과로 발생	명령어 실행과 무관하게 발생
발생위치	CPU 내부	CPU외부의 하드웨어 장치
예시	0으로 나누기, 오버 플로, 잘못된 메모리 접근	키보드 입력, 마우스 클릭, 하드디스크 완료
관련용어	소프트웨어 인터럽트, 내부 인터럽트	하드웨어 인터럽트, 외부 인터럽트
처리 방법	예외처리 루틴, 시그널 핸들러	인터럽트 서비스 루틴(ISR)

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 시그널의 종류와 번호

표 12-5 유닉스의 주요 시그널과 특징

시그널	번호	기본동작	특징
SIGHUP	1	종료	hangupm 터미널 접속이 끊어짐
SIGINT	2	종료	<code>Ctrl+C</code> , 사용자 인터럽트
SIGQUIT	3	종료(코어 덤프)	<code>Ctrl+\</code> , 디버깅용 종료
SIGILL	4	종료(코어 덤프)	잘못된 명령어 실행
SIGABRT	6	종료(코어 덤프)	abort 함수 호출 시 발생, 비정상 종료
SIGKILL	9	종료	무조건 종료, 무시 불가능
SIGALRM	14	종료	알람 타이머 완료 시
SIGTERM	15	종료	일반적인 종료 요청
SIGCHLD	17	무시	자식 프로세스가 종료 시에 부모에게 전달
SIGCONT	18	재시작	정지된 프로세스를 다시 실행
SIGSTOP	19	정지	일시 정지, 무시 불가능, SIGCONT로 재시작 가능
SIGTSTP	20	정지	<code>Ctrl+Z</code> , 터미널 일시 정지, SIGCONT로 재시작 가능

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 시그널의 종류와 번호

```
$ sleep 100
^Z          <- SIGSTP 발생, 일시 정지
[1]+ Stopped sleep 100
$ bg        <- SIGCONT 발생, 재시작
[1]+ sleep 100 &
```

그림 12-33 유닉스의 사용자 시그널

- SIGSTP 시그널 발생하여 sleep 프로세스를 일시정지
- SIGCONT 시그널 발생하여 sleep 프로세스를 재시작

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 시그널의 종류와 번호

```
$ sleep 100
^Z          <- SIGSTP로 일시 정지
[1]+ Stopped  sleep 100
$ ps
  PID TTY  TIME  CMD
  2171 pts/0 00:00:00 bash
  2329 pts/0 00:00:00 sleep
  2381 pts/0 00:00:00 ps
$ kill 2329      <- SIGINT로 죽지 않음
$ kill -9 2329   <- SIGKILL은 무조건 종료
[1]+ killed      sleep 100
```

그림 12-34 shell 프로세스에서의 프로세스 종료

- 터미널(shell 프로세스)에서는 kill 명령어로 프로세스에 SIGINT를 보냄
- 일시 정지 중인 프로세스이면 kill -9 명령어 사용

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 시그널 핸들러 재지정 코드

코드 12-1 시그널 핸들러 재지정 코드

```
01 #include <stdio.h>
02 #include <unistd.h>
03 #include <signal.h>
04
05 void sig_handler(int signum) {           // 시그널 핸들러 함수 본체
06     printf("\nNumber: %d signal received", signum);
07 }
08
09 void main() {
10     void (*handler)(int);                // 함수 포인터 변수 선언
11
12     handler = signal(SIGINT, sig_handler); // SIGINT 핸들러 지정
13
14     printf("Waiting signal\n");
15     pause();                             // 시그널 수신까지 대기
16 }
```

Waiting signal

^C

Number: 2 signal received

<- 시그널 핸들러 실행

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 기본 핸들러 재지정

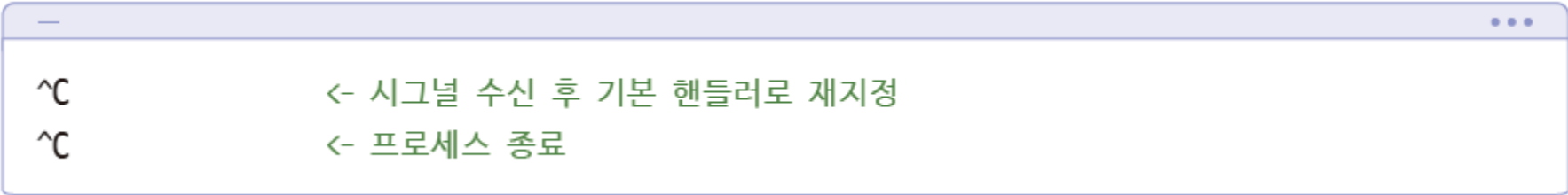
코드 12-2 기본 핸들러 재지정 코드

```
01  #include <stdio.h>
02  #include <signal.h>
03
04  void sig_handler(int signum) {           // 시그널 핸들러 함수 본체
05      void (*handler)(int);              // 함수 포인터 변수 선언
06
07      handler = signal(SIGINT, NULL);      // SIGINT 기본 핸들러 재지정
08  }
09
10  void main() {
11      void (*handler)(int);              // 함수 포인터 변수 선언
12
13      handler = signal(SIGINT, sig_handler); // SIGINT 핸들러 지정
14      while(1);                          // 무한 루프
15  }
```

03 입출력 시스템

3. [심화] 인터럽트 종류와 시그널 처리

◆ 기본 핸들러 재지정



```
^C          <- 시그널 수신 후 기본 핸들러로 재지정
^C          <- 프로세스 종료
```

Thank You!